

Prompt 3 — Colores por Grupo en Planning + Nombre Huésped en Celdas de Grupo

El objetivo

Cada grupo tiene un color único asignado al crearlo. Las celdas del planning que pertenecen a ese grupo se muestran con ese color. Adentro de cada celda aparece el nombre del huésped (no "GRP").

PARTE 1 — Agregar campo `color` a la tabla `groups`

1a — En `shared/schema.ts`

```
// En la tabla groups, agregar:  
color: text("color").default("#6366f1"), // índigo por defecto
```

1b — Migración

```
await db.execute(sql`  
  ALTER TABLE groups ADD COLUMN IF NOT EXISTS color TEXT DEFAULT '#6366f1'  
`);
```

1c — Actualizar tipo `InsertGroup` y `Group`

Drizzle los infiere automáticamente con el campo agregado en la tabla.

PARTE 2 — Selector de color en `GroupFormDialog`

2a — Paleta de colores predefinidos

En el formulario de grupo (`GroupFormDialog`), agregar después del campo "Nombre":

```

<div>
  <Label>Color del grupo</Label>
  <div className="flex gap-2 flex-wrap mt-2">
    {[
      "#6366f1", // índigo
      "#8b5cf6", // violeta
      "#ec4899", // rosa
      "#ef4444", // rojo
      "#f97316", // naranja
      "#eab308", // amarillo
      "#22c55e", // verde
      "#06b6d4", // cyan
      "#3b82f6", // azul
      "#84cc16", // lima
    ]}.map((color) => (
      <button
        key={color}
        type="button"
        className={`w-7 h-7 rounded-full border-2 transition-transform hover:scale-110 ${
          formData.color === color ? "border-foreground scale-110" : "border-transparent"
        }`}
        style={{ backgroundColor: color }}
        onClick={() => setFormData({ ...formData, color })}
        data-testid={`button-color-${color.replace("#", "")}`}
        title={color}
      />
    )))
  </div>
</div>

```

Agregar `color: group?.color || "#6366f1"` al estado inicial de `formData`.

PARTE 3 — Pasar `groupId` y `groupColor` en el planning data

3a — En `server/storage.ts`, método `getPlanningData` (versión DB)

En `reservationsMap`, agregar `groupId` y `groupColor`:

```

// Buscar donde se hace groupsMap y agregar groupColor al buscar el grupo:
let groupName: string | undefined;
let groupId: string | undefined;
let groupColor: string | undefined;

```

```

if (isGroupReservation) {
  const link = allGroupLinks.find(l => l.reservationId === res.id);
  if (link) {
    const group = groupsMap.get(link.groupId);
    if (group) {
      groupName = group.name;
      groupId = group.id;
      groupColor = group.color || "#6366f1";
    }
  }
}

reservationsMap[res.id] = {
  // ... campos existentes ...
  isGroup: isGroupReservation,
  groupName,
  groupId,          // ← agregar
  groupColor,       // ← agregar
  // ...
};

```

Hacer lo mismo en la implementación **in-memory** del mismo método.

3b — Actualizar el tipo en el frontend

En `planning.tsx`, el tipo local de reservación del planning (probablemente inferido del API response) necesita los nuevos campos. Si hay un tipo explícito, agregar:

```

type PlanningReservation = {
  // ... campos existentes ...
  groupId?: string;
  groupColor?: string;
};

```

PARTE 4 — Usar el color del grupo en las celdas del planning

4a — Nueva función `getGroupColor`

En `planning.tsx`, agregar:

```

function getGroupCellStyle(groupColor: string | undefined): React.CSSProperties {
  if (!groupColor) return {};
  // Convertir hex a RGB para poder usar con opacidad

```

```

const hex = groupColor.replace("#", "");
const r = parseInt(hex.substring(0, 2), 16);
const g = parseInt(hex.substring(2, 4), 16);
const b = parseInt(hex.substring(4, 6), 16);
return {
  backgroundColor: `rgba(${r}, ${g}, ${b}, 0.25)`,
  borderColor: `rgba(${r}, ${g}, ${b}, 0.7)`,
};
}

```

4b — Aplicar en la celda del planning

Buscar el `DraggableReservationCell` y modificar el `className/style` para grupos:

```

<DraggableReservationCell
  id={`drag-${reservationId}-${room.id}-${day}`}
  reservationId={reservationId!}
  roomId={room.id}
  onClick={() => handleCellClick(room, day, status, reservationId)}
  className={`h-8 rounded border flex items-center justify-center transition-all
    reservation.isGroup
      ? "" // sin clase de color, usar style inline
      : getSourceColor(reservation.source) // color por fuente solo para no-gru
  `}
  style={reservation.isGroup ? getGroupCellStyle(reservation.groupColor) : undefi
  data-testid={`cell-${room.id}-${day}`}
>

```

4c — Mostrar nombre del huésped (no “GRP”) en celdas de grupo

Buscar la línea que muestra el texto dentro de la celda:

```

// Reemplazar:
{reservation.isGroup ? "GRP" : reservation.guestName.split(" ")[0]}

// Por:
{reservation.guestName.split(" ")[0]}

```

Los grupos ya tienen `guestName` en el `planning data` (es el nombre del huésped asignado a esa reserva). Si dice “Sin Asignar” (cuando la habitación del grupo no tiene huésped asignado todavía), mostrar abreviación del grupo:

```

{reservation.guestName === "Sin Asignar" || !reservation.guestName
  ? reservation.groupName?.substring(0, 4).toUpperCase() || "GRP"
}

```

```
: reservation.guestName.split(" ")[0]}
```

4d — Actualizar la leyenda

En `Legend`, la leyenda de “Origen” aplica solo a reservas no-grupales. Para grupos, el color viene del grupo mismo. Agregar una nota en la leyenda:

```
<div className="flex items-center gap-2 mt-1">
  <span className="text-xs text-muted-foreground italic">
    Los grupos se muestran con su color asignado individual
  </span>
</div>
```

PARTE 5 — Mostrar color en la lista de grupos

En `GroupsPage`, en la tabla de grupos, agregar un indicador de color en la primera columna (junto al código):

```
<TableCell>
  <div className="flex items-center gap-2">
    <div
      className="w-3 h-3 rounded-full flex-shrink-0"
      style={{ backgroundColor: group.color || "#6366f1" }}
      title={`Color del grupo: ${group.color}`}
    />
    <span className="font-mono text-sm">{group.groupCode}</span>
  </div>
</TableCell>
```

Resumen de archivos modificados

Archivo	Cambio
shared/schema.ts	Campo <code>color</code> en tabla <code>groups</code>
server/migrate.ts	ALTER TABLE <code>groups</code>
server/storage.ts	<code>groupId</code> y <code>groupColor</code> en <code>reservationsMap</code> (ambas implementaciones)

<code>client/src/pages/planning.tsx</code>	<code>getGroupCellStyle</code> , celdas con color de grupo, nombre huésped en lugar de "GRP"
<code>client/src/pages/groups.tsx</code> (GroupFormDialog)	Selector de color, <code>color</code> en formData
<code>client/src/pages/groups.tsx</code> (tabla)	Indicador de color en lista